



Database After Graduation

Week 2

**By Narges Yarahmadi
Feb 2026**

Table of Contents

1. Introduction: SQL in the Real World	3
2. Production-Level SQL: What Makes It Different?	3
3. Query Behavior and Execution	4
4. The Cost of Full Table Scans	4
5. Execution Plans and EXPLAIN	5
6. Indexing Strategies	6
Types of Indexes	6
Composite Indexes	6
7. Read-Heavy vs Write-Heavy Systems	7
Read-Heavy Systems	7
Write-Heavy Systems	7
8. Join Optimization.....	7
9. Query Anti-Patterns	8
10. Schema Design and Performance	8
11. Locking and Concurrency.....	9
12. Scaling Beyond a Single Database	9
13. Monitoring and Observability	10
14. Case Study: E-Commerce Backend	10
15. Final Mindset: Think in Systems	11
Hands-On Demonstration: Understanding Indexing in MySQL	12
1. Step 1: Create the Database and Base Table (Without Secondary Indexes)	12
2. Step 2: Populate the Table with a Large Dataset	12
3. Step 3: Execute a Query Without an Index	13
4. Step 4: Add a Secondary Index	13
5. Step 5: Demonstrate a Composite Index	14
6. Step 6: Demonstrate a Common Index Misuse	14
7. Step 7 (Optional): Measure Execution Time	15
8. Key Takeaways from This Exercise	16
Hands-On Demonstration: production case study	17
Writing Production-Level SQL Queries	17
Query Execution Behavior	20

1. Introduction: SQL in the Real World

In academic settings, SQL is evaluated primarily on correctness. If a query produces the expected output for a small dataset, it is considered successful. However, in production systems, correctness is only the beginning.

Production SQL must operate under real-world constraints:

- Large datasets (millions to billions of rows)
- Concurrent users
- Limited hardware resources
- Strict latency requirements
- Reliability and uptime expectations
- Financial cost implications

A query that is correct but inefficient can degrade system performance, increase infrastructure costs, and cause cascading failures in distributed systems.

The objective of this session is to shift perspective from “writing SQL that works” to “designing queries and schemas that behave well under load.”

2. Production-Level SQL: What Makes It Different?

Production-level SQL differs from academic SQL in four fundamental ways:

1. It must scale.
2. It must be predictable.
3. It must minimize resource consumption.
4. It must align with system architecture.

In real backend systems, SQL queries are rarely isolated. They are embedded inside APIs, services, background workers, analytics pipelines, and transactional workflows.

When a user clicks a button in a web application, that action may trigger:

- A read query to fetch data
- A write query to update state
- Multiple joins
- Aggregations
- Logging inserts
- Cache invalidations

Each query contributes to overall system performance. Poorly designed SQL accumulates technical debt quickly.

Therefore, production SQL is not only about retrieving data; it is about preserving system stability.

3. Query Behavior and Execution

When MySQL receives a SQL query, it does not execute it line by line in the order written. Instead, it parses the query, transforms it into an internal representation, and creates an execution plan.

An execution plan determines:

- Which indexes to use
- Join order
- Access method (full table scan vs index lookup)
- Sorting strategy
- Temporary table usage

Understanding execution behavior is critical because the database engine may execute a query in a way you did not expect.

For example, consider:

```
SELECT * FROM orders WHERE user_id = 42;
```

If there is no index on `user_id`, MySQL performs a full table scan. That means it reads every row in the `orders` table, even if only a few rows match.

On a table with 10 rows, this is harmless.

On a table with 100 million rows, this is catastrophic.

4. The Cost of Full Table Scans

A full table scan means the database reads every row in the table sequentially. This is expensive in terms of:

- Disk I/O
- Memory usage
- CPU time

Full table scans are sometimes unavoidable, but they should never be accidental.

Common causes include:

- Missing indexes
- Using functions on indexed columns
- Leading wildcards in LIKE queries
- Type mismatches

Example:

```
SELECT * FROM users WHERE email LIKE '%gmail.com';
```

Even if email is indexed, this query cannot use the index efficiently because the wildcard at the beginning prevents ordered lookup.

In production systems, accidental full table scans are one of the most common performance issues.

5. Execution Plans and EXPLAIN

The EXPLAIN keyword allows us to inspect how MySQL plans to execute a query.

Example:

```
EXPLAIN SELECT * FROM orders WHERE user_id = 42;
```

The output reveals:

- Access type
- Possible indexes
- Key used
- Rows examined
- Extra operations

Important access types include:

- ALL (full table scan)
- ref (index lookup)
- range (range scan)
- const (single row match)

If EXPLAIN shows type = ALL, that is a warning sign for large tables.

Using EXPLAIN should be standard practice when optimizing queries.

6. Indexing Strategies

An index is a data structure that allows the database to locate rows efficiently without scanning the entire table.

Think of an index as a phone book. Without it, you would have to read every entry to find a name.

Types of Indexes

In MySQL, the most common index type is a B-tree index. B-tree indexes are efficient for:

- Equality comparisons
- Range queries
- Ordered retrieval

Example:

```
CREATE INDEX idx_user_id ON orders(user_id);
```

This allows fast lookup of orders by user_id.

Composite Indexes

Composite indexes include multiple columns.

Example:

```
CREATE INDEX idx_user_status ON orders(user_id, status);
```

This index can efficiently serve queries filtering by:

- user_id
- user_id and status together

But not:

- status alone

Order matters in composite indexes.

This is known as the leftmost prefix rule.

Understanding how composite indexes work is essential for production systems with complex filtering patterns.

7. Read-Heavy vs Write-Heavy Systems

System design must consider workload type.

Read-Heavy Systems	Write-Heavy Systems
Examples: • Analytics dashboards • News platforms • Content feeds Characteristics: • Frequent SELECT queries • Rare writes Optimization strategies: • Aggressive indexing • Denormalization • Materialized views • Caching layers Indexes improve read performance but increase write cost.	Examples: • Logging systems • Financial transactions • Messaging platforms Characteristics: • Frequent INSERTs and UPDATEs Optimization strategies: • Fewer indexes • Batch writes • Append-only design • Partitioning Every index adds overhead to writes because the database must update the index structure whenever data changes.

There is always a trade-off.

8. Join Optimization

Joins are essential in relational databases, but they can become expensive.

Example:

```
SELECT users.name, orders.total
FROM users
JOIN orders ON users.id = orders.user_id;
```

Performance depends on:

- Indexes on join columns
- Join order
- Data distribution

If orders.user_id is not indexed, the join may require scanning the entire orders table for each user.

In production, joins should always be supported by indexes on join keys.

9. Query Anti-Patterns

Certain patterns commonly degrade performance:

1. SELECT *
Retrieves unnecessary columns.
Increases network payload and memory usage.
2. N+1 Queries
Running one query to fetch items, then another query per item.
Common in ORM misuse.
3. Unbounded Queries
Missing LIMIT on large result sets.
4. Implicit Type Conversion
Comparing string to integer.
Prevents index usage.

Avoiding these patterns is part of writing production-grade SQL.

10. Schema Design and Performance

Schema design directly impacts performance.

Normalized schemas reduce redundancy but increase joins.

Denormalized schemas reduce joins but increase storage and complexity.

Example:

In an e-commerce system:

Option 1: Store product price in products table.

Option 2: Copy product price into order_items table at purchase time.

The second approach increases redundancy but improves historical accuracy and simplifies queries.

Design decisions must consider business logic and workload patterns.

11. Locking and Concurrency

Databases must handle concurrent operations.

When multiple users update the same data, locking mechanisms ensure consistency.

However, locks can reduce throughput.

Example:

```
UPDATE inventory SET quantity = quantity - 1 WHERE product_id = 10;
```

If many users purchase simultaneously, this row becomes a contention hotspot.

Solutions include:

- Optimistic locking
- Sharding
- Caching counters

Concurrency management is a key part of production database design.

12. Scaling Beyond a Single Database

As systems grow, a single database may not be sufficient.

Common scaling strategies:

Vertical scaling:

Increase hardware resources.

Horizontal scaling:

Add replicas.

Read replicas:
Offload read queries.

Sharding:
Partition data across multiple nodes.

Application-level decisions influence database scaling options.

13. Monitoring and Observability

Production databases must be monitored.

Key metrics include:

- Query latency
- Slow query logs
- CPU utilization
- Disk I/O
- Lock wait times

Observability allows engineers to detect issues before users notice.

Performance tuning is continuous, not a one-time task.

14. Case Study: E-Commerce Backend

Consider an online store.

Core tables:

- users
- products
- orders
- order_items

Common queries:

- Fetch product listings
- Retrieve user order history
- Update inventory
- Generate sales reports

Design considerations:

- Index product category for browsing.
- Index user_id in orders.
- Use composite indexes for frequent filters.
- Avoid heavy joins in high-traffic endpoints.

Each endpoint must be analyzed for query cost and concurrency impact.

15. Final Mindset: Think in Systems

Production SQL is not about syntax mastery.

It is about understanding:

- Data access patterns
- Execution behavior
- Trade-offs between reads and writes
- Hardware limitations
- Business requirements

A database is not a passive storage system. It is an active participant in system performance.

Every index is a design decision.

Every join is a cost.

Every query is a system event.

When engineers think this way, databases become powerful tools rather than bottlenecks.

Hands-On Demonstration: Understanding Indexing in MySQL

This practical exercise demonstrates how indexing affects query execution behavior and performance in a relational database. The objective is to observe the difference between a full table scan and an index-based lookup using real data and execution plans.

1. Step 1: Create the Database and Base Table (Without Secondary Indexes)

Begin by creating a dedicated database for this demonstration and defining a table that initially contains no indexes except the primary key.

```
CREATE DATABASE week2_demo;
USE week2_demo;

CREATE TABLE orders (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    status VARCHAR(20),
    total DECIMAL(10,2),
    created_at DATETIME
);
```

At this stage, the table contains only one index: the primary key on `id`. No indexes exist on `user_id`, `status`, or `created_at`.

This setup allows us to observe how MySQL behaves when no supporting indexes are available.

2. Step 2: Populate the Table with a Large Dataset

To clearly observe performance differences, the table must contain a substantial number of rows. The following stored procedure inserts 100,000 records with randomized values.

```
DELIMITER //

CREATE PROCEDURE insert_orders()
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= 100000 DO
        INSERT INTO orders (user_id, status, total, created_at)
        VALUES (
            FLOOR(1 + RAND()*1000),
            IF(RAND() > 0.5, 'completed', 'pending'),
            ROUND(RAND()*1000, 2),
            NOW()
        );
        i = i + 1;
    END WHILE;
END //
```

```

        RAND()*500,
        NOW() - INTERVAL FLOOR(RAND()*365) DAY
    );
    SET i = i + 1;
END WHILE;
END //

DELIMITER ;

CALL insert_orders();

```

After execution, the `orders` table contains approximately 100,000 rows. This volume is sufficient to demonstrate execution plan differences.

3. Step 3: Execute a Query Without an Index

Now analyze how MySQL executes a query filtering by `user_id`.

```
EXPLAIN SELECT * FROM orders WHERE user_id = 42;
```

Focus on the following columns in the output:

- `type`
- `key`
- `rows`

You will likely observe:

- `type = ALL`
- `key = NULL`
- `rows ≈ 100000`

This indicates a full table scan. MySQL must read every row in the table to locate matching records because no index exists on `user_id`.

In large production tables, this behavior becomes extremely costly.

4. Step 4: Add a Secondary Index

Now create an index on `user_id`.

```
CREATE INDEX idx_user_id ON orders(user_id);
```

Run the same execution plan again:

```
EXPLAIN SELECT * FROM orders WHERE user_id = 42;
```

This time you should observe:

- type = ref
- key = idx_user_id
- rows significantly smaller than before

The database is no longer scanning the entire table. Instead, it uses the B-tree index structure to directly locate the matching rows.

Conceptually, this reduces the lookup complexity from $O(n)$ to approximately $O(\log n)$.

5. Step 5: Demonstrate a Composite Index

Consider a more realistic query involving multiple filtering conditions.

```
EXPLAIN
SELECT *
FROM orders
WHERE user_id = 42
AND status = 'completed';
```

With only the `user_id` index, MySQL can partially optimize the query, but it still needs to filter by `status` afterward.

Now create a composite index:

```
CREATE INDEX idx_user_status ON orders(user_id, status);
```

Execute the EXPLAIN statement again.

You will observe that MySQL can now use both columns within the index structure. This demonstrates the importance of column order in composite indexes and introduces the Leftmost Prefix Rule:

A composite index can be used efficiently when the query filters using the leading column(s) in the defined index order.

6. Step 6: Demonstrate a Common Index Misuse

Consider the following query:

```
EXPLAIN
```

```
SELECT *
FROM orders
WHERE YEAR(created_at) = 2024;
```

Even if an index exists on `created_at`:

```
CREATE INDEX idx_created_at ON orders(created_at);
```

You will likely observe:

- `type = ALL`

This occurs because applying a function (`YEAR`) to the indexed column prevents MySQL from using the index efficiently. The database must compute the function for every row before applying the filter.

Now rewrite the query in an index-friendly form:

```
EXPLAIN
SELECT *
FROM orders
WHERE created_at >= '2024-01-01'
AND created_at < '2025-01-01';
```

In this case, MySQL can use the index because the query operates directly on the raw column values.

This example illustrates an important production rule:

Avoid applying functions to indexed columns in WHERE clauses.

7. Step 7 (Optional): Measure Execution Time

To observe execution time differences, enable profiling:

```
SET profiling = 1;

SELECT * FROM orders WHERE user_id = 42;

SHOW PROFILES;
```

Compare execution time before and after adding indexes.

While the dataset is relatively small, measurable differences should appear. In larger production systems, these differences become dramatically significant.

8. Key Takeaways from This Exercise

When an index is created:

- MySQL builds a B-tree structure
- Data becomes logically organized for efficient lookup
- Query execution can shift from full table scans to index-based access

However, indexes are not free.

Each additional index:

- Consumes storage
- Increases INSERT and UPDATE cost
- Adds maintenance overhead

Effective database engineering requires balancing read performance against write cost.

Understanding how execution plans change after indexing is a foundational skill for backend system design.

Hands-On Demonstration: production case study

Production SQL & Query Execution

In this session, we move beyond “SQL that works” and focus on **SQL that survives production**. Real systems are not judged by whether a query returns correct rows, but by whether it does so **efficiently, reliably, and predictably under load**. This class is about learning how databases actually execute your queries and how small design choices can dramatically affect performance.

We will explore how the database engine thinks, how it plans query execution, and how developers can guide that process using indexes and query structure.

Writing Production-Level SQL Queries

In production systems, SQL queries must be readable, maintainable, and performant. A query may run thousands or millions of times per day, so clarity and efficiency matter more than clever tricks.

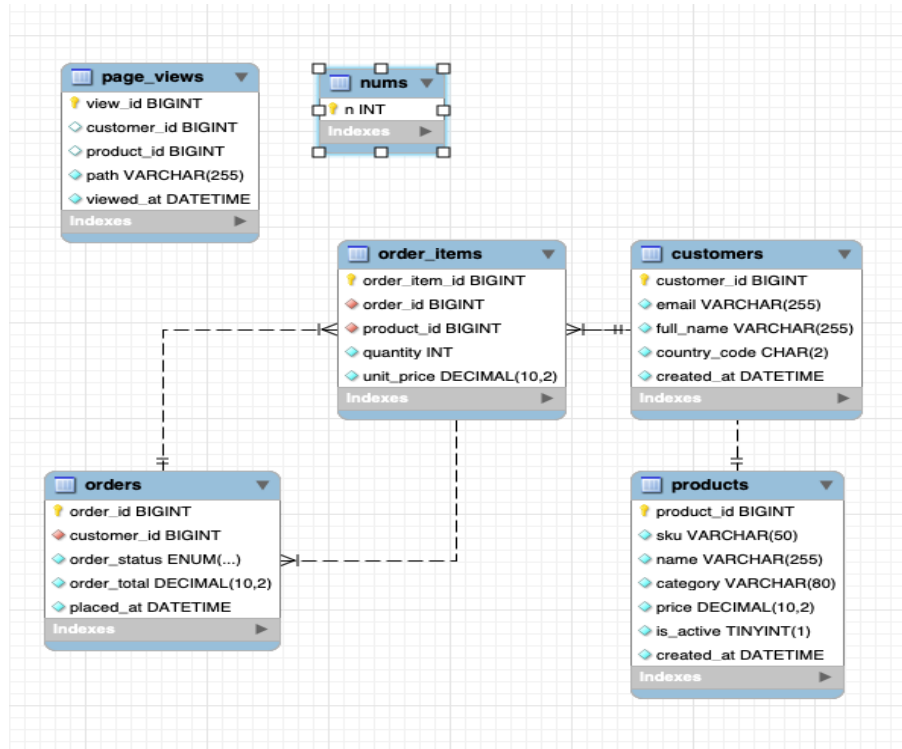
We will discuss how to write SQL that:

- Uses clear aliases and consistent formatting
- Avoids unnecessary complexity
- Selects only required columns instead of using `SELECT *`
- Handles filtering, aggregation, and joins in a predictable way
- Scales well as data volume grows

We will also compare “developer-friendly” queries with “database-friendly” queries and learn how to balance readability with performance.

Let’s look into some samples:

Go to bright space and download the `Creat_db_Insert_to_db.sql` from week2 material. Run it. The dataset ER diagram would be as follow:



let's start working on the following questions.

A) "Top customers by revenue in last 30 days"

Bad version (still correct, but heavier than needed)

```

EXPLAIN
SELECT c.customer_id, c.email, SUM(o.order_total) AS revenue
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
WHERE o.placed_at >= NOW() - INTERVAL 30 DAY
      AND o.order_status IN ('PAID','SHIPPED')
GROUP BY c.customer_id, c.email
ORDER BY revenue DESC
LIMIT 20;
  
```

Better version (filters early and only selects necessary columns)

```

EXPLAIN
SELECT o.customer_id, SUM(o.order_total) AS revenue
FROM orders o
WHERE o.placed_at >= NOW() - INTERVAL 30 DAY
      AND o.order_status IN ('PAID','SHIPPED')
GROUP BY o.customer_id
ORDER BY revenue DESC
LIMIT 20;
  
```

Then join only the top 20 to customers

```

EXPLAIN
SELECT c.customer_id, c.email, t.revenue
FROM (
  SELECT o.customer_id, SUM(o.order_total) AS revenue
  FROM orders o
  WHERE o.placed_at >= NOW() - INTERVAL 30 DAY
  AND o.order_status IN ('PAID','SHIPPED')
  GROUP BY o.customer_id
  ORDER BY revenue DESC
  LIMIT 20
) t
JOIN customers c ON c.customer_id = t.customer_id
ORDER BY t.revenue DESC;

```

Index upgrade for this pattern

```

-- Helps WHERE on status + placed_at and GROUP BY customer_id
CREATE INDEX idx_orders_status_placed_customer
ON orders(order_status, placed_at, customer_id);

```

“Find customers who bought category X but never bought category Y”

This is a classic production query used in marketing, recommendation, churn analysis.

```

EXPLAIN
SELECT DISTINCT o.customer_id
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.order_status IN ('PAID','SHIPPED')
  AND p.category = 'phone'
  AND o.customer_id NOT IN (
    SELECT o2.customer_id
    FROM orders o2
    JOIN order_items oi2 ON oi2.order_id = o2.order_id
    JOIN products p2 ON p2.product_id = oi2.product_id
    WHERE o2.order_status IN ('PAID','SHIPPED')
      AND p2.category = 'laptop'
  );

```

Better production version using NOT EXISTS (more reliable than NOT IN with NULLs)

```

EXPLAIN
SELECT DISTINCT o.customer_id
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.order_status IN ('PAID','SHIPPED')
  AND p.category = 'phone'
  AND NOT EXISTS (
    SELECT 1
    FROM orders o2
    JOIN order_items oi2 ON oi2.order_id = o2.order_id

```

```
JOIN products p2 ON p2.product_id = oi2.product_id
WHERE o2.customer_id = o.customer_id
      AND o2.order_status IN ('PAID','SHIPPED')
      AND p2.category = 'laptop'
);
```

Index upgrades for this pattern

```
CREATE INDEX idx_products_category_id ON products(category, product_id);
CREATE INDEX idx_orders_customer_status ON orders(customer_id, order_status,
order_id);
CREATE INDEX idx_items_order_product ON order_items(order_id, product_id);
```

Query Execution Behavior

SQL is declarative: you describe **what** you want, not **how** to get it. The database engine decides how to execute your query internally.

In this part, we will explain:

- How MySQL parses and optimizes a query
- How join order is chosen
- How filtering and aggregation are applied
- Why two logically identical queries can have very different performance
- What happens when indexes exist versus when they do not

Understanding execution behavior helps you reason about slow queries instead of guessing or relying on trial and error.